

OSS-Next — Blockers Report

2026-07-23 · Branch `integrate-wp-render`

What is blocked, who can unblock it, and exactly what is needed.

One fix unblocks the most: the Django checkout crash (B1). Braintree credentials are now configured and a real sandbox charge has been verified end to end. That means *every remaining obstacle to a working purchase is on the backend*, and three further items are waiting behind this one.

Status Since the 2026-07-21 Audit

Closed since that PDF was exported — if you are reading `AUDIT_TASK_LIST_2026-07-21.pdf` alongside this, these three no longer apply:

- **Client-controlled charge amount** — the payment endpoint accepted the price from the browser, so a crafted request could buy a \$3,000 container for a cent. The total is now recomputed server-side and the browser's figure is never charged.
- **Charged-but-no-order** — customers now get a dedicated "payment received, order needs attention" screen with the payment reference, instead of a raw error beside a live Place Order button that would charge them twice.
- **Deprecated `middleware.ts`** — renamed to `proxy.ts` for Next 16, with the homepage A/B test verified still running in production.

Owner	Open items	Nature
Backend team	6	Code fixes and one ownership question
Client	5	Business/product decisions, no engineering blocked behind them
Frontend	0	Nothing actionable is outstanding

Backend Blockers

B1 · CRITICAL Checkout endpoint crashes on every call

POST `/api/orders/checkout` returns 500 on every single request:

```
ValueError: Currency formatting is not possible using the 'C' locale.  
app/orders/views.py, line 89  
    locale.currency(float(item.total), grouping=True)
```

The server process has no locale configured, so Python's `locale.currency()` always raises. **The order record is saved before the crash** — we confirmed real orders being created despite every attempt returning 500. With payment now live, a customer would have their card charged, have an order created, and still be shown a failure.

What we need: call `locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')` at startup, or drop `locale.currency()` and format the amount directly (e.g. `f"${amount:,.2f}"`). The second is more robust — it does not depend on locales being installed in the container image.

B2 Order status never becomes `paid`

We send `payment_method: "braintree"` and the Braintree `transaction_id` with every checkout, but the order status does not move to `paid`. Something else server-side governs it, and we cannot see what.

What we need: confirm what sets an order to `paid`, and whether `transaction_id` is actually persisted. If it is being discarded, tracing a payment from an order record back to Braintree will not be possible.

B3 The pages API serves malformed CSS

The CSS minifier strips `/* */` delimiters but leaves the comment text in place, and emits unbalanced braces:

```
once slick adds its class,show it smoothly .shipping-container-slider...{...}  
}.slick-slide:not(.slick-active){opacity:0!important}
```

Strict parsers reject this outright. **Browsers error-recover by discarding every rule after the fault**, which means the live WordPress site is silently losing styling today — this is not only our problem. We work around it with a tolerant parser, but the source is wrong.

What we need: fix the minifier in the Django CSS pipeline so comments are removed whole and braces stay balanced.

B4 Guest abandoned-cart notifications cannot succeed

POST `/api/abandoned-carts/create` requires `abandoned_cart_id` plus a full billing address. Guests never receive a `cart_id` at all (`/api/cart/*` is authenticated-only), and by definition a guest has given us nothing but an email address. So the call can never validate, no matter what the front end sends. Our capture flow is built and working; it has nothing valid to post to.

What we need — a decision, either is fine: relax validation to accept an email-only notification when there is no `abandoned_cart_id`, or let guest carts mint a `cart_id` the way logged-in carts do.

B5 New reviews never appear in the list endpoint

Creating a review succeeds (200, full object returned), and a duplicate is correctly rejected. But `GET /api/reviews/list` returns `count: 0` for that product — site-wide, product-filtered, and authenticated alike. This looks like moderation, but it is not documented anywhere. It blocks two features from ever being exercised: edit-prefill of an existing review, and the populated state of the reviews carousel.

What we need: confirm whether reviews require approval, where that is done, and ideally approve one or two so the populated UI can be tested.

B6 Who sends order confirmation emails?

There is no email-sending code anywhere in the Next.js application — no SendGrid, nodemailer, or SMTP integration. If confirmation emails are being sent today, it is entirely a Django responsibility and invisible from our side. We would rather confirm than assume it is covered.

What we need: a yes/no on whether Django sends order confirmation emails, and on which events.

Client Decisions

None of these block engineering work — they block *starting* it. Each is waiting on a business answer rather than a technical one. Two carry active customer-facing risk (C1, C2).

C1 · CUSTOMER-FACING RISK The homepage quote form silently discards submissions

The form displays "✓ **Quote Request Sent — We'll be in touch!**" while the customer's name, email, phone, ZIP and container details go **nowhere**. Nothing is stored and nothing is sent. Every lead submitted through it is lost, and the visitor believes they have contacted the business.

What we need: the Zoho Forms account and form details, so this can be wired to the real destination. **Until then, consider disabling the form or removing the success message** — the current behaviour is worse than having no form at all.

C2 · CUSTOMER-FACING RISK The site advertises live chat that does not exist

The homepage trust strip promises "Phone, Email & Chat" support. There is no chat feature anywhere in the application, and none planned as yet.

What we need: either a decision to build chat, or approval to change that line of copy. **The copy fix takes minutes and needs no engineering decision** — we recommend it now regardless of the longer-term answer on chat.

C3 Newsletter signup UI

The largest genuinely unbuilt feature. The backend calls already exist on our side, so each piece is "build the UI and call this": a footer widget, an account-dashboard toggle, and an unsubscribe page.

What we need: confirmation to build it, and where the signup should appear.

C4 Address book — one address, or many?

Today a customer has exactly one billing and one shipping address, stored on their profile. There is no add/remove, no multiple saved addresses, and no default selection. This is fine as "edit your address", but it is not an address book.

What we need: a decision on whether multiple saved addresses are wanted. Note this also needs backend support — the profile endpoint has no concept of more than one.

C5 Shipment tracking in order history

Order history shows real order status (pending / paid / shipped / delivered), but there is no carrier or tracking number anywhere in the data. Customers cannot track a delivery.

What we need: confirmation that tracking numbers are wanted, and whether the backend can begin returning a carrier and tracking number on the order record.

Waiting Behind B1

These are not separate work items — they unblock themselves once the checkout crash is fixed:

Item	Why it is stuck
------	-----------------

End-to-end purchase verification	Everything else is built and verified; the endpoint 500s.
Populated order-history rendering	Built, but no order can be created to test it against.
Cart-close on successful checkout	Wired, but never reached, because checkout never completes.

Known and Deliberate — No Action Requested

Product-listing images do not set a priority hint. The cards render through the search library's own list component, which does not expose the position of an item, so marking the first card would mean replacing that component with a hand-rolled version and risking pagination and empty-state regressions. The gain is a minor performance hint on content that is rendered after page load anyway. Recorded here so it is not re-reported as an oversight.